MPOO 2027-1 · GRUPO 06

# Parcial 1 · correcciones

Versión B del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

**Antes de empezar.** Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_esquivel_iker
git pull --no-rebase
```

**Qué tienes que hacer.** En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/esquivel_iker/examen/p1/`, con tu push de siempre.

## Pregunta 0.1 · Preguntas abiertas

**Pregunta**

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
En la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

**Tu respuesta**

```
Los atributos son las cualidades con las que cuenta el objeto, por ejemplo una calculadora,
tiene botones, color y componentes dentro de si, el metodo es lo que sabe hacer, por ejemplo
sumar, restar e infinidad de operaciones matematicas.
```

**Corrección.** Bien: el atributo es una cualidad o parte del estado (botones, color) y el método es lo que el objeto sabe hacer (sumar, restar). Los ejemplos de la calculadora son coherentes.

## Pregunta 0.2 · Preguntas abiertas

**Pregunta**

¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

**Tu respuesta**

```
Para que el ejecutable sea siempre el mismo y no pueda recibir modificaciones.
```

**Corrección.** Incorrecta: se confunde static con final. main es el punto de entrada desde donde arranca la ejecución, y se declara static para poder invocarlo desde la clase sin crear antes una instancia.

## Pregunta 0.3 · Preguntas abiertas

**Pregunta**

¿Qué diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

**Tu respuesta**

```
Un atributo de estancia es aqueol que solo posee un objeto, y uno static pertenece a toda a la clase de objeto, lo que podrian compartir dos objetos de la misma clase, como ejemplo una marca de zapatos, comparten el mismo modelo, pero el color que pueden tener no.
```

**Corrección.** Correcta: el atributo de instancia es propio de cada objeto y el static pertenece a la clase y lo comparten todos sus objetos. El ejemplo del modelo compartido frente al color de cada zapato ilustra bien la diferencia.

## Pregunta 0.4 · Preguntas abiertas

**Pregunta**

¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

**Tu respuesta**

```
Una variable de tipo primitivo guarda el valor que permite el tipo de dato, como el int, guarda el valor entero que le asignemos a la variable, por ejemplo int a =1;, si le dieramos un valor decimal, en este caso 3.5 solo guardaria el entero, es decir 3.
Una de referencia puede almacenar valores que incluso incluyen null, como los arreglos, pero al momento de compilar si no creamos referencias no nulas a los valores del arreglo nos puede saltar un error
```

**Corrección.** La parte del primitivo es correcta (int a = 1 guarda el valor directamente), pero asignar 3.5 a un int no guarda 3: no compila sin un cast. Faltó la idea central de la referencia: guarda una referencia a un objeto (por ejemplo, un String o un arreglo), no el objeto mismo; poder valer null no la define.

## Pregunta 0.5 · Preguntas abiertas

**Pregunta**

¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

**Tu respuesta**

```
javac compila el archivo.java, y la JVM crea el archivo .class.
```

**Corrección.** Javac sí compila el .java, pero es javac quien produce el bytecode en el archivo .class, no la JVM. La JVM ejecuta ese bytecode.

## R1 · ¿Qué imprime?

**Pregunta**

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[] datos = {7, 5, 3};
int[] alias = datos;
int i = 0;
while (i < 3) {
    System.out.println(alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println(mitad);
```

**Tu respuesta**

```
B79
3.0
```

> **Corrección.** La línea B79 es correcta: reasignar alias no cambia datos. Faltan las tres primeras líneas (3, 2, 1, de dividir 7, 5 y 3 entre 2 en el ciclo). La última es 2.0, no 3.0, porque 5/2 se trunca a 2 antes del cast; el formato decimal sí estaba bien.

**Qué escribir a mano.** La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

## R2 a) · ¿Compila, truena o funciona?

**Pregunta**

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int modo = 1;
switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

**Tu respuesta**

**Clasificación**

```
Funciona
```

**Causa o salida**

```
Funciona pero al no poner "break;" en "case 1:" sigue cayendo y ejecuta los demas cases de abajo. Salida:/LISTO/FIN
```

> **Corrección.** Correcto: funciona e imprime LISTO y FIN, porque sin break la ejecución sigue al case siguiente.

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R2 b) · ¿Compila, truena o funciona?

**Pregunta**

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
{ int nivel = 4; }
System.out.println(nivel);
```

**Tu respuesta**

**Clasificación**

```
No compila
```

**Causa o salida**

```
Se esta declarando un int con una sitaxis de forma incorrecta
```

**Corrección.** La clasificación 'No compila' es correcta. La causa señala la declaración del int nivel, pero no es un error de sintaxis, porque la declaración está bien escrita. Es un error de ámbito: nivel solo existe dentro de las llaves donde se declaró, y fuera de ellas no se reconoce.

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R2 c) · ¿Compila, truena o funciona?

**Pregunta**

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

**Tu respuesta**

**Clasificación**

```
Compila pero truena
```

**Causa o salida**

```
Compila pero null es una palabra reservada
```

**Corrección.** La clasificación compila pero truena es correcta y se señala bien que null tiene que ver con el error. La causa, sin embargo, no es que null sea palabra reservada, sino que se llama trim() sobre una referencia null, que no apunta a ningún objeto (NullPointerException).

**Qué escribir a mano.** La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

## R3 · Modelado: base, reto y justificación

**Pregunta**

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

**Tu respuesta**

**Clase**

```
class Cargador{
    static int capacidad = 4;
    int n=0;
    static final int ranuras = 4;
    Cargador c = new Cargador(n);
}
void colocar(capacidad,n){
    if(n<capacidad){
        System.out.println("Hay ranuras disponibles")
        n+=1;
        return true;
    }else{
        system.out.println("No hay ranuras disponibles")
        return false;
}

Reto:
Static es la capacidad, porque es para cualquier cargador, pero puede depender si ya hay
baterias en el o no.
Final son fabricados porque es el estado en el que estan inicialmente sin que pasen por alguna
modificacion futura.
```

**Justificaciones**

```
El tipo static int de capacidad es porque es universal para los cargadores aunque pueden estar
vacios. El static int final son las ranuras que si o si tienen todos los cargadores
independientemente si tienen baterias o no. Para crear un cargador solo necesitamos saber el
numero de baterias que se encuentran en el. Para colocar una bateria dentro del cargador
preguntamos si el numero de pilas que hay dentro es menor al de su capacidad, si es asi, el
numero de baterias dentro del cargador se suma 1, sino no es posible colocar una nueva
bateria.
```

**Corrección.** Bien: n es un int de instancia mutable que cuenta las ocupadas, ranuras es una constante y colocar incrementa el contador. En contra: capacidad debía ser final de instancia (cada cargador fija la suya), no static; falta el contador static fabricados; no hay constructor (el campo Cargador c = new Cargador(n) no lo sustituye); colocar quedó fuera de la clase, sin tipos en los parámetros y declarado void aunque devuelve boolean; y falta el método de consulta. El reto quedó invertido: capacidad es final y fabricados es static, y ninguno necesita ambos.

## R4 · Código con repetición

**Pregunta**

El arreglo intentos contiene exactamente tres códigos enteros, en el orden en que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null. No lo modifiques.

```
static int intentoValido(int[] intentos, int clave)
```

Ejemplos (argumentos → retorno):

{13, 42, 8}, 42 → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3}, 42 → 0

Escribe el método completo. No imprimas ni leas del teclado.

**Tu respuesta**

```
static int intentoValido(int[] intentos, int clave){
    int correcto = 42;
    do{
        for(clave =1; clave <intentos.length+1; clave++){
            for(i=1; i<intentos.length+1; i++){
                if(intento[]==correcto){
                    System.out.println(intentos[i] +"->"+ clave);
                }else{
                    System.out.println(intentos[i]+"-> 0");
                }
              }
            }
    }while(intento[]=!clave);
}

//Comentarios: Mi idea aqui es recorrer el arreglo de intentos, mientras eso el valor de clave
iniciado en 1 va a aumentar hasta que sea menor que el valor de la longitud del arreglo+1,
para asi leer todos los valores, igual mientars un valor i va a recorrecr el mismo arreglo,
cuando el avlor del indice coincida con el correcto nos arrojara el valor y la clave
correspondiente, sino nos arrojara el valor de la clave 0, todo eso mientras el vlor del
indice del intento sea diferente al de la clave.
```

**Corrección.** La cabecera es correcta. Usa do-while con repetición sobre el arreglo. La lógica falla: compara contra un 42 fijo en lugar de clave, reutiliza clave como contador, imprime en vez de devolver y no se detiene en el primer acierto. Además, los índices de 1 a length dejan fuera intentos[0] y piden intentos[3]. Se reconoce de forma parcial numerar desde 1 y la idea del 0 al fallar. Lo esperado: recorrer i de 0 a 2 dentro del do-while, devolver i + 1 si intentos[i] == clave y devolver 0 después del ciclo.

**Qué escribir a mano.** El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

## R5 · Código con selección

**Pregunta**

Devuelve la acción del módulo con switch sobre Modo. Primero: si bateria < 10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

```
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
static String accion(Modo modo, int bateria)
```

| Regla o valor recibido | Texto devuelto |
|---|---|
| APAGADO | "DETENER" |
| MEDIR | "LEER" |
| CALIBRAR | "AJUSTAR" |
| PRUEBA | "SIMULAR" |

**Tu respuesta**

```
switch(Modo){
    Case 1:
    System.out.println("
}
```

**Corrección.** Respuesta incompleta: solo se inicia un switch, que es la forma pedida, pero va sobre el tipo Modo y con case 1. Faltan la cabecera static String accion(Modo modo, int bateria), devolver CARGAR si bateria < 10 antes que todo, devolver SIN MODO si modo es null, y los case APAGADO/MEDIR/CALIBRAR/PRUEBA con return DETENER/LEER/AJUSTAR/SIMULAR.

**Qué escribir a mano.** El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.